AFRA: correção demonstrada e a conjectura do tempo

Versão final revisada: teorema de correção e término, limite exponencial e conjectura Las Vegas

Luiz Guilherme*
Lunz Research Group

26 de outubro de 2026

Resumo

Especificamos formalmente um procedimento de decisão para SAT, aqui designado pela sigla AFRA (não expandida no manuscrito original), baseado em simplificações preservadoras de satisfatibilidade, memoização exata de fórmulas residuais normalizadas e ramificação exaustiva com escolhas aleatorizadas. Definimos uma instância concreta de referência, AFRA_0, e um sistema explícito de hipóteses (H1)–(H6) que confina o conjunto de variáveis ativas, impõe um orçamento global de tamanho às fórmulas intermediárias e limita uniformemente o custo local por nó. Sob essas hipóteses demonstramos que o procedimento decide qualquer fórmula CNF corretamente, com erro zero para toda fita aleatória, visitando no máximo $2^{n+1} - 1$ nós, e que seu tempo e espaço de pior caso são limitados por $q(|F|) \cdot 2^n$. Em seguida isolamos a questão em aberto: se o tempo computacional esperado (equivalentemente, o número esperado de resíduos distintos visitados) pode ser limitado polinomialmente em 3-CNF. Formulamos essa questão como uma conjectura do tipo Las Vegas para um algoritmo fixado, deduzimos suas consequências em teoria da complexidade ($\mathsf{NP} = \mathsf{RP} = \mathsf{ZPP}$, colapso de PH a NP, refutação da variante aleatorizada da ETH) com os quantificadores corretos, discutimos por que limites inferiores de resolução tornam a conjectura improvável, e propomos metas intermediárias e um protocolo de avaliação empírica. Nenhum experimento foi realizado neste trabalho; o texto deve ser lido como nota metodológica e de especificação formal.

1 Teorema ou conjectura?

A resposta é: os dois, aplicados a afirmações diferentes.

- **Correção e término** são afirmações demonstráveis para um procedimento especificado; entram como teorema (Teorema 3.3).
- **Tempo polinomial** não tem prova nem refutação conhecida para a instância AFRA_0 aqui definida; entra como conjectura (Conjectura 4.1).

Duas observações orientam a formulação. Primeiro, como a ramificação é exaustiva e as simplificações preservam satisfatibilidade, o procedimento não comete erro para *nenhuma* fita aleatória r: a aleatoriedade afeta apenas a ordem em que variáveis e ramos são explorados. Assim, uma formulação do tipo "probabilidade de erro no máximo 1/3" seria trivialmente satisfeita e não captura a dificuldade. Segundo, toda a dificuldade concentra-se no tempo, o que sugere a formulação Las Vegas (sempre correto, tempo aleatório), e não a formulação BPP.

Registramos desde já, seguindo o parecer de revisão, que o resultado de correção e término *não é específico* do AFRA: ele é a demonstração padrão para qualquer busca por ramificação com simplificações equissatisfatíveis e cache válido. A contribuição deste texto é metodológica: fixar um algoritmo concreto, enunciar hipóteses suficientes de forma precisa, separar correção, término e limite de tempo, e situar corretamente a conjectura de eficiência diante da literatura.

* O autor utilizou assistentes de inteligência artificial (GPT-6 Astra, Gemini 4 Argon e Claude) como apoio à revisão do texto. A responsabilidade pelo conteúdo, pelas demonstrações e pelas conclusões é integralmente do autor.

2 Definições, especificação operacional e hipóteses

2.1 Convenções e notação

Seja $\mathcal{V} = \{x_1, \ldots, x_n\}$ um universo finito e ordenado de variáveis booleanas. Um *literal* ℓ é uma variável x_i ou sua negação $\neg x_i$; escrevemos $\mathrm{var}(\ell) = x_i$. Uma *cláusula* é um conjunto finito de literais (interpretado como disjunção); uma fórmula em forma normal conjuntiva (CNF) é um conjunto finito de cláusulas (interpretado como conjunção), $F = C_1 \wedge \cdots \wedge C_m$. A cláusula vazia é denotada por $\square$ (sempre falsa) e a conjunção vazia por $\top = \emptyset$ (sempre verdadeira). Dizemos que $F \in k$-CNF se toda cláusula $C \in F$ satisfaz $|C| \leq k$; em particular, adotamos a convenção de que "3-CNF" significa largura *no máximo* três, de modo que 2-CNF $\subseteq$ 3-CNF.

Para uma fórmula CNF G definimos:

- $\mathrm{Var}(G) = \bigcup_{C \in G} \{\mathrm{var}(\ell) : \ell \in C\}$, o conjunto de variáveis que ocorrem em G. Para a entrada F, supomos, sem perda de generalidade (renomeando variáveis), que $\mathrm{Var}(F) = \mathcal{V}$ e escrevemos $n = |\mathrm{Var}(F)|$.
- $\|G\| = \sum_{C \in G} |C|$, o número de ocorrências de literais.
- $|G|$, o *tamanho* de G: o comprimento em bits da codificação fixa em que cada cláusula é uma lista de literais, cada literal ocupando $L = \lceil \log_2(2n+2) \rceil$ bits, seguida de um separador de L bits; assim $|G| = L \sum_{C \in G} (|C| + 1)$. Como cada variável de $\mathcal{V}$ ocorre em F, vale $n \leq \|F\| \leq |F|$. Remover cláusulas ou literais de G nunca aumenta $|G|$.
- $D(G) \in \{0, 1\}$, o predicado de satisfatibilidade: $D(G) = 1$ se existe atribuição $\alpha : \mathrm{Var}(G) \to \{0, 1\}$ que satisfaz todas as cláusulas de G, e $D(G) = 0$ caso contrário. Duas fórmulas G, G' são *equissatisfatíveis* se $D(G) = D(G')$.
- Para $x \in \mathcal{V}$ e $b \in \{0, 1\}$, a *restrição sintática* $G|_{x=b}$ é obtida removendo de G todas as cláusulas que contêm o literal tornado verdadeiro por $x = b$ e removendo o literal complementar das demais cláusulas. Vale $\mathrm{Var}(G|_{x=b}) \subseteq \mathrm{Var}(G) \setminus \{x\}$ e $|G|_{x=b}| \leq |G|$.

2.2 Normalização sintática

Fixamos a ordem total sobre literais $x_1 \prec \neg x_1 \prec x_2 \prec \neg x_2 \prec \cdots \prec x_n \prec \neg x_n$. A rotina $\mathrm{Norm}(G)$ (i) remove toda cláusula tautológica (que contém x_i e $\neg x_i$); (ii) em cada cláusula restante, remove literais duplicados e ordena os literais segundo $\prec$; (iii) remove cláusulas duplicadas e ordena a lista de cláusulas lexicograficamente. A saída $\mathrm{Norm}(G)$ é uma cadeia canônica para o conjunto de cláusulas resultante, logicamente equivalente a G, com $\mathrm{Var}(\mathrm{Norm}(G)) \subseteq \mathrm{Var}(G)$ e $|\mathrm{Norm}(G)| \leq |G|$. O custo é $O(|G| \log |G|)$ por ordenação por comparação.

Observação 2.1 (critério de reutilização). Neste artigo adotamos a igualdade sintática de representações normalizadas, $\mathrm{Norm}(G_1) = \mathrm{Norm}(G_2)$, como critério *suficiente* e computacionalmente barato para reutilização segura de resultados. Esse critério não é necessário em geral: também seria seguro reutilizar resultados sob equivalência ou equissatisfatibilidade comprovada (por exemplo, sob renomeação bijetiva de variáveis), mas tal teste não é trivialmente polinomial e não é adotado aqui. Semelhança heurística, por outro lado, *não* autoriza reutilização.

2.3 Modelo computacional e a família AFRA

O modelo é uma máquina de acesso aleatório (RAM) probabilística, com custo logarítmico por palavra, equipada com uma fita aleatória $r \in \{0, 1\}^\omega$ de bits independentes e uniformes, lida da esquerda para a direita. Escrevemos $\mathbb{P}_r$ e $\mathbb{E}_r$ para a medida de probabilidade correspondente.

Um algoritmo A da *família AFRA* é determinado por duas sub-rotinas: uma rotina de simplificação $\mathrm{Simplify}_A$ e uma heurística de escolha Choose_A. O procedimento recursivo Solve_A é o

mesmo para toda a família e é descrito a seguir. O cache global $\mathcal{M}$ é inicializado vazio e implementado como uma árvore binária de busca balanceada indexada pela cadeia H (comparação lexicográfica de cadeias).

Procedimento $\mathrm{Solve}_A(G, U)$**.** Recebe uma fórmula CNF G e um conjunto $U \subseteq \mathcal{V}$ de variáveis disponíveis, com $\mathrm{Var}(G) \subseteq U$. Na raiz, chama-se $\mathrm{Solve}_A(F, \mathrm{Var}(F))$.

P1. $H \leftarrow \mathrm{Simplify}_A(G)$.

P2. Se $\square \in H$, devolve 0. Se $H = \top$, devolve 1.

P3. Se existe $(H, v) \in \mathcal{M}$, devolve v.

P4. $(x, b) \leftarrow \mathrm{Choose}_A(H, r)$, com $x \in \mathrm{Var}(H)$ e $b \in \{0, 1\}$.

P5. $v_1 \leftarrow \mathrm{Solve}_A(H|_{x=b},\, U \setminus \{x\})$. Se $v_1 = 1$, insere $(H, 1)$ em $\mathcal{M}$ e devolve 1.

P6. $v_2 \leftarrow \mathrm{Solve}_A(H|_{x=1-b},\, U \setminus \{x\})$. Insere (H, v_2) em $\mathcal{M}$ e devolve v_2.

2.4 A instância de referência AFRA_0

Fixamos um parâmetro inteiro $\tau \geq 1$ (por exemplo, $\tau = 10$).

Rotina $\mathrm{Simplify}_0(G)$**.**

S1. $\widehat{G} \leftarrow \mathrm{Norm}(G)$.

S2. *Propagação unitária e literal puro até ponto fixo.* Enquanto $\square \notin \widehat{G}$ e existir em $\widehat{G}$ uma cláusula unitária $\{\ell\}$ ou um literal puro ℓ (isto é, ℓ ocorre em $\widehat{G}$ e $\neg\ell$ não ocorre): seja $x = \mathrm{var}(\ell)$ e b o valor que torna ℓ verdadeiro; faça $\widehat{G} \leftarrow \mathrm{Norm}(\widehat{G}|_{x=b})$.

S3. *Eliminação de variável por resolução com orçamento.* Percorra as variáveis $x \in \mathrm{Var}(\widehat{G})$ em ordem crescente de índice. Sejam P_x e N_x os conjuntos de cláusulas de $\widehat{G}$ contendo x e $\neg x$, respectivamente. Se $|P_x| \leq \tau$ e $|N_x| \leq \tau$, calcule o conjunto R_x de todos os resolventes sobre x entre cláusulas de P_x e N_x e a candidata

$$G' = \mathrm{Norm}\big((\widehat{G} \setminus (P_x \cup N_x)) \cup R_x\big).$$

Se $|G'| \leq |\widehat{G}|$, faça $\widehat{G} \leftarrow G'$ e volte a S2. Se nenhuma variável produzir candidata aceita, prossiga.

S4. Devolva $\widehat{G}$.

Heurística $\mathrm{Choose}_0(H, r)$**.** Seja $V_H = \mathrm{Var}(H)$ e seja $W \subseteq V_H$ o conjunto das variáveis de frequência máxima de ocorrência em H, com $k = |W| \geq 1$. Lê-se $\lceil \log_2 k \rceil$ bits de r como um inteiro i e escolhe-se a $(i \bmod k)$-ésima variável de W em ordem de índice; em seguida lê-se um bit b de r como polaridade inicial. O consumo de bits é finito e fixo por nó; a distribuição resultante sobre W não é exatamente uniforme quando k não é potência de 2, o que é irrelevante para os resultados de correção e término.

2.5 Medidas de complexidade

Para um algoritmo A da família, entrada F e fita r, distinguimos:

- $N_A(F, r)$: o número de *nós visitados*, isto é, de invocações de Solve_A;
- $R_A(F, r)$: o número de *resíduos distintos processados*, isto é, de invocações que chegam ao passo P4 (nós de ramificação);

- $T_A(F, r)$: o *tempo computacional* total, em operações elementares da RAM, até a parada (ou ∞ se não houver parada);
- $S_A(F, r)$: o *espaço* máximo utilizado, em bits.

Essas grandezas não são intercambiáveis; as relações entre elas são estabelecidas no Lema 3.1 e na Observação 4.2.

2.6 Hipóteses

Os resultados gerais aplicam-se a qualquer algoritmo A da família cujas sub-rotinas satisfaçam as hipóteses abaixo. As hipóteses (H3) e (H4) descrevem o procedimento Solve_A, fixo para toda a família, e são listadas por clareza.

(H1) *Simplificações preservam satisfatibilidade.* Para toda fórmula CNF G, $H = \text{Simplify}_A(G)$ satisfaz $D(H) = D(G)$ e $H = \text{Norm}(H)$.

(H2) *Confinamento de variáveis.* Para toda G, $\text{Var}(\text{Simplify}_A(G)) \subseteq \text{Var}(G)$; nenhuma variável nova é introduzida e nenhuma variável eliminada é reintroduzida. A heurística devolve sempre $x \in \text{Var}(H)$.

(H3) *Memoização exata de estados concluídos.* O cache $\mathcal{M}$ inicia vazio e só recebe pares (H, v) ao final de uma chamada que calculou v para a fórmula normalizada H. A reutilização dá-se apenas por igualdade sintática exata da chave.

(H4) *Semântica exaustiva com curto-circuito.* Casos base: 0 se $\square \in H$, 1 se $H = \top$. Um ramo só deixa de ser explorado se o ramo irmão devolveu 1; o valor 0 só é devolvido após ambos os ramos devolverem 0.

(H5) *Orçamento global de tamanho.* Existe um polinômio $s(\cdot)$ tal que toda fórmula G ou H que ocorra em qualquer nó da execução de $A(F, r)$, para toda r, satisfaz $|H| \leq s(|F|)$.

(H6) *Término determinístico e custo local uniformemente polinomial.* Existe um polinômio $t_{\text{loc}}(\cdot)$ tal que, em todo nó da execução de $A(F, r)$ e para toda fita r, a execução completa de Simplify_A, a heurística Choose_A (que consome um número finito de bits de r) e as operações de consulta e inserção em $\mathcal{M}$ terminam em tempo total no máximo $t_{\text{loc}}(|F|)$.

Observação 2.2. As hipóteses (H2), (H5) e (H6) substituem as hipóteses originais (H3) e (H4), que eram insuficientes: a mera equissatisfatibilidade não impede a introdução de variáveis auxiliares (destruindo o progresso da recursão), e a polinomialidade de cada rodada de simplificação *no tamanho corrente* não impede crescimento composto superpolinomial ao longo da profundidade (por exemplo, $s_{i+1} \leq s_i^2$ produz $s_i \leq s_0^{2^i}$). O orçamento global (H5) e o custo local uniforme (H6) são medidos em relação ao tamanho da *entrada original* $|F|$.

3 Resultados demonstrados

Lema 3.1 (estrutura da recursão)**.** *Sob (H2), para toda entrada F com n variáveis e toda fita r: toda chamada $\text{Solve}_A(G, U)$ satisfaz $\text{Var}(G) \subseteq U$; cada chamada filha recebe um conjunto disponível de cardinalidade estritamente menor; a árvore de chamadas tem profundidade no máximo n e no máximo $2^{n+1} - 1$ nós. Além disso, cada nó de ramificação insere em $\mathcal{M}$ uma chave distinta, de modo que $R_A(F, r) = |\mathcal{M}|$ ao final e*

$$R_A(F, r) \;\leq\; N_A(F, r) \;\leq\; 2R_A(F, r) + 1.$$

Demonstração. Na raiz, $\mathrm{Var}(F) = U$. Se $\mathrm{Var}(G) \subseteq U$, então por (H2) $\mathrm{Var}(H) \subseteq \mathrm{Var}(G) \subseteq U$, a variável escolhida satisfaz $x \in \mathrm{Var}(H) \subseteq U$ e $\mathrm{Var}(H|_{x=b}) \subseteq \mathrm{Var}(H) \setminus \{x\} \subseteq U \setminus \{x\}$; logo a invariante propaga-se e $|U|$ diminui em exatamente 1 por nível. Quando $|U| = 0$ tem-se $\mathrm{Var}(H) = \emptyset$, de modo que toda cláusula de H é vazia: ou $H = \top$ ou $\square \in H$, e a chamada para em P2. Assim a profundidade é no máximo n e, como cada nó tem no máximo dois filhos, há no máximo $\sum_{i=0}^{n} 2^i = 2^{n+1} - 1$ nós.

Para a segunda parte: uma chave H não pode ocorrer como nó de ramificação aninhado dentro de outro nó de ramificação com a mesma chave, pois $x \in \mathrm{Var}(H)$ é removida de U e os descendentes têm variáveis em $U \setminus \{x\}$. Dois nós de ramificação não aninhados com a mesma chave não podem ocorrer: o primeiro a terminar insere H em $\mathcal{M}$ antes de o segundo executar P3, que então devolveria o valor armazenado sem ramificar. Logo cada nó de ramificação insere uma chave nova e $R_A = |\mathcal{M}|$ ao final. Em uma árvore em que apenas os R_A nós de ramificação têm filhos (no máximo dois cada), o número total de nós é no máximo $2R_A + 1$. □

Lema 3.2 (ramificação com simplificação)**.** *Seja H uma fórmula CNF, $x \in \mathrm{Var}(H)$ e, para $b \in \{0,1\}$, $G_b = H|_{x=b}$ e $H_b = \mathrm{Simplify}_A(G_b)$. Sob (H1),*

$$D(H) = D(G_0) \vee D(G_1) = D(H_0) \vee D(H_1).$$

Demonstração. Uma atribuição α satisfaz H se, e somente se, $\alpha(x) = 0$ e α restrita a $\mathrm{Var}(H)\setminus\{x\}$ satisfaz G_0, ou $\alpha(x) = 1$ e a mesma restrição satisfaz G_1: as cláusulas eliminadas em G_b são exatamente as satisfeitas pelo literal fixado, e o literal removido das demais é falso sob $x = b$. Reciprocamente, toda atribuição satisfatória de G_b estende-se com $x = b$ a uma atribuição satisfatória de H. Logo $D(H) = D(G_0) \vee D(G_1)$. Por (H1), $D(G_b) = D(H_b)$ para cada b, o que conclui a prova. Note-se que não se exige que $\mathrm{Simplify}_A$ preserve o conjunto de modelos, apenas a satisfatibilidade. □

Teorema 3.3 (correção e término)**.** *Sob (H1)–(H6), para toda fórmula CNF F com n variáveis e para* toda *fita $r \in \{0,1\}^\omega$, a execução de $\mathrm{Solve}_A(F, \mathrm{Var}(F))$ termina, visita no máximo $N_A(F, r) \leq 2^{n+1} - 1$ nós e devolve $D(F)$. Em particular, a probabilidade de erro é identicamente zero.*

Demonstração. Término. Pelo Lema 3.1, a árvore de chamadas tem no máximo $2^{n+1} - 1$ nós. Por (H6), o processamento local de cada nó termina em tempo finito para toda fita r. Logo a execução termina após um número finito $K \leq 2^{n+1} - 1$ de chamadas concluídas.

Correção. Ordenemos as chamadas pela ordem em que *concluem*, $j = 1, \ldots, K$, e denotemos a j-ésima por $\mathrm{Solve}_A(G_j, U_j)$ com $H_j = \mathrm{Simplify}_A(G_j)$. Provamos, por indução forte em j, a invariante conjunta:

(i) após a conclusão da j-ésima chamada, todo par $(H', v) \in \mathcal{M}$ satisfaz $v = D(H')$;

(ii) a j-ésima chamada devolve $D(G_j)$.

Suponha (i) e (ii) válidas para todas as chamadas concluídas antes de j (para $j = 1$, o cache está vazio e a hipótese é vazia). Por (H1), $D(H_j) = D(G_j)$. Há três casos.

Caso base (P2). Se $\square \in H_j$, toda atribuição falsifica $\square$, logo $D(H_j) = 0$ e a chamada devolve $0 = D(G_j)$. Se $H_j = \top$, toda atribuição a satisfaz, $D(H_j) = 1$ e a chamada devolve $1 = D(G_j)$. O cache não é alterado, preservando (i).

Acerto no cache (P3). Existe $(H_j, v) \in \mathcal{M}$. Por (H3), esse par foi inserido ao final de alguma chamada anteriormente concluída; por (i) aplicada à chamada concluída imediatamente antes de j, $v = D(H_j) = D(G_j)$. O cache não é alterado.

Ramificação (P4–P6). Escolhe-se $x \in \mathrm{Var}(H_j)$. A primeira chamada filha, sobre $H_j|_{x=b}$, conclui antes de j e, por (ii), devolve $v_1 = D(H_j|_{x=b})$. Se $v_1 = 1$, pelo Lema 3.2 $D(H_j) = 1$; a chamada insere $(H_j, 1)$, preservando (i), e devolve $1 = D(G_j)$. Se $v_1 = 0$, a segunda filha, sobre

$H_j|_{x=1-b}$, conclui antes de j e devolve $v_2 = D(H_j|_{x=1-b})$; pelo Lema 3.2, $D(H_j) = 0 \vee v_2 = v_2$; a chamada insere (H_j, v_2), preservando (i), e devolve $v_2 = D(G_j)$.

A chamada raiz é a última a concluir ($j = K$) e devolve $D(F)$. Como o argumento vale para toda fita r, a probabilidade de erro é 0. □

Proposição 3.4 (limite exponencial de tempo e espaço)**.** *Sob (H1)–(H6), existe um polinômio $q(\cdot)$ tal que, para toda fórmula CNF F com n variáveis e toda fita r,*

$$T_A(F, r) \leq q(|F|) \cdot 2^n \qquad e \qquad S_A(F, r) \leq q(|F|) \cdot 2^n.$$

Demonstração. Pelo Teorema 3.3, $N_A(F, r) \leq 2^{n+1} - 1 < 2 \cdot 2^n$. Por (H6), o custo total de cada nó (simplificação completa, normalização, heurística, consulta e inserção no cache) é no máximo $t_{\text{loc}}(|F|)$, medido em relação à entrada original. Somando sobre os nós,

$$T_A(F, r) \leq N_A(F, r) \cdot t_{\text{loc}}(|F|) \leq 2\, t_{\text{loc}}(|F|) \cdot 2^n.$$

Quanto ao espaço: por (H5), toda chave armazenada tem tamanho no máximo $s(|F|)$ e há no máximo $R_A \leq 2^{n+1}$ chaves, cada uma com valor de um bit e ponteiros de $O(n)$ bits na árvore balanceada; a pilha de recursão tem profundidade no máximo $n + 1$ com quadros de tamanho $O(s(|F|) + |F|)$. Usando $n \leq |F|$, basta tomar $q(|F|) = 2\, t_{\text{loc}}(|F|) + c\,(|F| + 2)\, s(|F|)$ para uma constante c adequada. □

Proposição 3.5 (conformidade de AFRA$_0$)**.** *O algoritmo* AFRA$_0$ *satisfaz (H1)–(H6), com $s(|F|) = |F|$ e $t_{\text{loc}}(|F|) = O(\tau^2 |F|^3 \log |F|)$.*

Demonstração. (H3) e (H4) são a estrutura de Solve. Para (H1) e (H2), verificamos cada regra de Simplify$_0$.

*Remoção de tautologias e duplicatas (*Norm*).* Cláusulas tautológicas são verdadeiras sob toda atribuição e cláusulas repetidas não alteram a conjunção; a fórmula resultante é logicamente equivalente. Nenhuma variável é introduzida.

Propagação unitária. Se $\{\ell\} \in \widehat{G}$, toda atribuição satisfatória torna ℓ verdadeiro; logo $D(\widehat{G}) = D(\widehat{G}|_{x=b})$, pelo mesmo argumento do Lema 3.2 com o ramo $x = 1 - b$ trivialmente insatisfatível (contém □).

Literal puro. Se ℓ ocorre e $\neg\ell$ não ocorre, qualquer atribuição satisfatória pode ser alterada para tornar ℓ verdadeiro sem falsificar cláusula alguma, pois nenhuma cláusula contém $\neg\ell$; essa atribuição modificada, restrita às demais variáveis, satisfaz $\widehat{G}|_{x=b}$. Reciprocamente, toda atribuição satisfatória de $\widehat{G}|_{x=b}$ estende-se com $x = b$. Logo $D(\widehat{G}) = D(\widehat{G}|_{x=b})$. A fórmula resultante é equissatisfatível, mas em geral não logicamente equivalente; para o procedimento, equissatisfatibilidade basta.

Eliminação por resolução. Pelo resultado clássico de Davis e Putnam [**?**], substituir $P_x \cup N_x$ pelo conjunto de todos os resolventes sobre x produz uma fórmula equissatisfatível (tautologias entre os resolventes podem ser descartadas, o que Norm faz). Os resolventes contêm apenas variáveis de $\text{Var}(\widehat{G}) \setminus \{x\}$.

Assim, (H1) e (H2) valem. Para (H5): Norm e as restrições sintáticas não aumentam o tamanho, e uma eliminação só é aceita se $|G'| \leq |\widehat{G}|$; como também $|H|_{x=b}| \leq |H|$, segue por indução na profundidade que toda fórmula intermediária satisfaz $|H| \leq |F|$.

Para (H6): cada passo de S2 e cada eliminação aceita em S3 removem ao menos uma variável de $\text{Var}(\widehat{G}) \subseteq \text{Var}(F)$, logo há no máximo n execuções de S2-passo e n eliminações aceitas, e portanto no máximo $n + 1$ varreduras completas de S3. Cada varredura examina no máximo n variáveis; para cada uma, calcula no máximo τ^2 resolventes de tamanho $O(|F|)$ e normaliza uma candidata de tamanho $O(\tau^2|F|)$ em tempo $O(\tau^2|F| \log |F|)$. Logo Simplify$_0$ termina em tempo $O(\tau^2 n^2 |F| \log |F|)$ para qualquer entrada do nó. A heurística Choose$_0$ conta ocorrências em tempo $O(|F|)$ e consome $\lceil \log_2 k \rceil + 1 \leq n + 1$ bits de r. Para o cache: pelo Lema 3.1 (que usa apenas (H2)), $\mathcal{M}$ tem no máximo 2^{n+1} chaves, cada uma de tamanho no máximo

$|F|$; uma consulta ou inserção em árvore balanceada custa $O(\log 2^{n+1}) = O(n)$ comparações de cadeias de comprimento $\leq |F|$, isto é, $O(n|F|)$. Somando e usando $n \leq |F|$, o custo local é $O(\tau^2|F|^3 \log |F|)$, um polinômio fixo em $|F|$, válido para toda fita r. □

Corolário 3.6. AFRA_0 *decide SAT para CNF arbitrária (em particular, 3-SAT) com erro zero para toda fita r, em tempo e espaço no máximo $q(|F|) \cdot 2^n$.*

Observação 3.7 (o que o Teorema 3.3 não diz). O Teorema 3.3 e a Proposição 3.4 estabelecem que AFRA_0 é um decisor exato e terminante, não que é rápido. A garantia $q(|F|)\cdot 2^n$ não é nova nem exclusiva: a busca por ramificação de Davis, Logemann e Loveland [**?**] (DPLL), com especificação análoga de simplificações e sem memoização, atinge tempo $O(|F| \cdot 2^n)$ com espaço *polinomial*. Convém distinguir historicamente: o procedimento de Davis e Putnam [**?**] é fundamentalmente uma eliminação sucessiva de variáveis por resolução, cujo espaço pode explodir se não for contido por um orçamento como o de S3; a ramificação em árvore é a contribuição de [**?**]. O uso de cache exato troca o espaço polinomial do DPLL por espaço potencialmente exponencial, $O(s(|F|)\cdot 2^n)$, no pior caso. Qualquer vantagem assintótica do AFRA deve, portanto, manifestar-se na redução do número esperado de resíduos visitados.

4 A conjectura

Conjectura 4.1 (Conjectura AFRA, formulação Las Vegas)**.** *Seja $A = \mathrm{AFRA}_0$ (ou outro algoritmo* fixado *da família, com heurística de custo polinomial, que satisfaça (H1)–(H6)). Existe um polinômio $p(\cdot)$ tal que, para toda $F \in$ 3-CNF,*

$$\mathbb{E}_r\big[T_A(F, r)\big] \leq p(|F|),$$

em que a esperança é tomada sobre a fita uniforme $r \in \{0,1\}^\omega$ e o mesmo polinômio p vale uniformemente para todas as entradas.

Observação 4.2 (resíduos distintos). Sob (H5)–(H6), para toda F e r vale $R_A(F, r) \leq N_A(F, r) \leq T_A(F, r) \leq t_{\mathrm{loc}}(|F|)\, N_A(F, r) \leq t_{\mathrm{loc}}(|F|)\,(2R_A(F, r)+1)$, pelo Lema 3.1 e porque cada nó consome ao menos uma operação. Tomando esperanças, a Conjectura 4.1 é *equivalente* à existência de um polinômio p_R tal que $\mathbb{E}_r[R_A(F, r)] \leq p_R(|F|)$ para toda $F \in$ 3-CNF. Essa reformulação envolve o número esperado de resíduos *efetivamente visitados*, não o número de resíduos possíveis de F; ela admite execuções raras de custo superpolinomial e não exige limite para toda fita r. O manuscrito anterior identificava, sem os quantificadores corretos, "quantidade de resíduos distintos polinomial para toda F" com a conjectura; a formulação correta é a acima.

Consequências, se a conjectura for verdadeira

1. $\mathsf{NP} \subseteq \mathsf{ZPP}$ *e colapso de* PH. Como A tem erro zero para toda fita (Teorema 3.3), a conjectura implica 3-SAT $\in \mathsf{ZPP}$. Pela NP-completude de 3-SAT sob reduções polinomiais, $\mathsf{NP} \subseteq \mathsf{ZPP}$. Como $\mathsf{ZPP} = \mathsf{RP} \cap \mathsf{coRP} \subseteq \mathsf{RP} \subseteq \mathsf{NP}$, obtém-se $\mathsf{NP} = \mathsf{RP} = \mathsf{ZPP}$; como ZPP é fechada por complemento, $\mathsf{coNP} = \mathsf{ZPP} = \mathsf{NP}$, e portanto

$$\mathsf{NP} = \mathsf{coNP} = \mathsf{RP} = \mathsf{coRP} = \mathsf{ZPP}, \qquad \mathsf{PH} = \mathsf{NP}.$$

   Isso *não* implica $\mathsf{P} = \mathsf{NP}$, pois não se sabe se $\mathsf{ZPP} = \mathsf{P}$.

2. *Versão de pior caso sobre a fita implica* $\mathsf{P} = \mathsf{NP}$. Se valer a hipótese mais forte de que existe p com $T_A(F, r) \leq p(|F|)$ para toda $F \in$ 3-CNF e *toda* fita r, então fixando uma fita trivialmente computável, por exemplo $r_0 = 000\ldots$, obtém-se o algoritmo determinístico $A(\cdot, r_0)$, de tempo polinomial, para 3-SAT; logo $\mathsf{P} = \mathsf{NP}$. (A fixação de "um r qualquer", como no manuscrito anterior, é imprecisa, pois uma fita infinita arbitrária pode não ser computável.)

3. *ETH e sua variante aleatorizada.* A hipótese de tempo exponencial (ETH, Impagliazzo e Paturi [**?**]) afirma que 3-SAT não admite algoritmo *determinístico* de tempo $2^{o(n)}$. A versão de pior caso do item anterior refutaria a ETH. A Conjectura 4.1 em esperança, por truncamento após $3p(|F|)$ passos e pela desigualdade de Markov, fornece um algoritmo aleatorizado de tempo polinomial com erro no máximo $1/3$, o que refuta a variante aleatorizada da ETH (que proíbe algoritmos aleatorizados com erro limitado em tempo $2^{o(n)}$). Sem teoremas de desaleatorização, que dependem de hipóteses de dureza, a versão Las Vegas *sozinha* não refuta a ETH determinística. O manuscrito anterior misturava as duas versões.

Por essas razões a conjectura deve ser tratada com ceticismo rigoroso: sua veracidade *implicaria* um avanço central em teoria da complexidade. Ela não é *equivalente* a tal avanço: $\mathsf{P} = \mathsf{NP}$ ou $\mathsf{NP} = \mathsf{ZPP}$ poderiam valer por meio de outros algoritmos enquanto AFRA_0, em particular, continua exponencial.

5 Por que a conjectura é difícil (e onde provavelmente falha)

5.1 Limites inferiores de resolução e simulação

A árvore de execução do DPLL clássico sobre uma fórmula insatisfatível corresponde a uma refutação por resolução em árvore. Com memoização de fórmulas residuais e eliminação por resolução, o sistema associado deixa de ser em árvore, mas permanece intimamente ligado à resolução geral. A formalização de sistemas de prova para DPLL com cache de fórmulas é objeto de Beame, Impagliazzo, Pitassi e Segerlind [**?**], cujas relações de simulação e separação em relação à resolução devem ser consultadas antes de qualquer afirmação sobre a força do cache.

O princípio de transferência pertinente é o seguinte: *se* uma variante A for p-simulável por resolução geral — isto é, se toda execução que refuta uma fórmula insatisfatível F em tempo t induzir uma refutação por resolução de F de tamanho $\mathrm{poly}(|F| + t)$ — então qualquer limite inferior superpolinomial para resolução sobre uma família de fórmulas 3-CNF refuta a Conjectura 4.1 para A, pois o tempo esperado domina o tempo mínimo sobre as fitas. Para AFRA_0, cada regra local é compatível com resolução (resolventes são passos de resolução; remover cláusulas de literal puro só reduz a fórmula a refutar; propagação unitária é resolução com cláusulas unitárias), mas a tradução de um nó UNSAT memoizado, resolvido sob um contexto de restrições ρ, em um lema reutilizável (a cláusula $\neg\rho$ derivada das cláusulas de F) exige um teorema de extração que *não demonstramos aqui*; ele é listado como meta na Seção **??**. Não afirmamos, portanto, que AFRA_0 seja p-simulável por resolução.

Caso a simulação seja estabelecida, as famílias clássicas relevantes são:

- *Princípio da casa dos pombos.* Haken [**?**] provou que a codificação usual de PHP_k^{k+1} ($k+1$ pombos, k casas), com $N = k(k+1) = \Theta(k^2)$ variáveis e cláusulas de largura k, exige refutações por resolução de tamanho $2^{\Omega(k)}$, isto é, $2^{\Omega(\sqrt{N})}$ em função do número de variáveis: superpolinomial, mas não $2^{\Omega(N)}$. Como a conjectura quantifica entradas 3-CNF, é preciso usar a versão com variáveis de extensão que decompõe cláusulas longas em cláusulas ternárias (ainda $\Theta(k^2)$ variáveis) e verificar explicitamente que o limite inferior se transfere para essa versão.

- *Fórmulas de Tseitin sobre expansores.* Urquhart [**?**] construiu fórmulas de paridade inconsistente sobre grafos expansores de grau limitado, que exigem refutações por resolução de tamanho $2^{\Omega(n)}$, em que n é o número de variáveis (arestas). Para grafos 3-regulares, cada restrição de paridade de grau 3 codifica-se em exatamente 4 cláusulas de largura 3, de modo que a família é nativamente 3-CNF com $m = \frac{8}{3}n$ cláusulas.